ESCUELA SUPERIOR POLITÉCNICA DE CHIMBORAZO

VICERRECTORADO ACADÉMICO

DIRECCIÓN DE DESARROLLO ACADÉMICO

FACULTAD: INFORMATICA Y ELECTRONICA

CARRERA: SOFTWARE

DISEÑO ARQUITECTÓNICO DE LA APLICACIÓN

CODIGO DE LA ASIGNATURA

SOF1TP49

APLICACIONES INFORMÁTICAS II

NOMBRES Y CÓDIGO

SHIRLEY EDIZA CHELA LLUMIGUANO

7322

1. Introducción

El presente documento describe el diseño arquitectónico de la aplicación móvil potenciada con machine learning para fortalecer la destreza speaking.

Para ello, se ha definido una arquitectura basada en la integración de los patrones MVVM, Repository, Microservicios y un enfoque orientado a eventos, los cuales permiten estructurar el sistema de manera modular, escalable y eficiente.

Este documento presenta la selección de la arquitectura, su representación mediante un diagrama arquitectónico y la justificación de las decisiones adoptadas en función de las necesidades del sistema.

2. Selección de la arquitectura

Para el desarrollo de la aplicación se ha seleccionado una arquitectura híbrida que integra los patrones MVVM (Model-View-ViewModel), Repository, Microservicios y un enfoque orientado a eventos. Esta combinación permite estructurar el sistema de forma organizada, facilitando la separación de responsabilidades y la comunicación eficiente entre sus componentes.

El patrón MVVM se implementa en la aplicación cliente desarrollada en Flutter, permitiendo separar la interfaz de usuario de la lógica de negocio y la gestión de datos. Por su parte, el patrón Repository actúa como intermediario entre el ViewModel y las distintas fuentes de datos, unificando el acceso tanto a servicios externos como a bases de datos.

La arquitectura de microservicios se utiliza en el backend, mediante un servidor independiente desarrollado en FastAPI, encargado de procesar tareas específicas como el reconocimiento de voz y el análisis gramatical. Finalmente, el enfoque orientado a eventos se implementa a través de Firebase, permitiendo la actualización automática de la información en tiempo real tanto para estudiantes como docentes.

Esta arquitectura permite que los diferentes componentes del sistema trabajen de manera conjunta, asegurando escalabilidad, mantenibilidad y una mejor experiencia de usuario.

3. Estilo arquitectónico

El sistema adopta principalmente un estilo arquitectónico distribuido, basado en la combinación de una arquitectura cliente-servidor y una arquitectura en capas.

Por un lado, se implementa el modelo cliente-servidor, donde la aplicación desarrollada en Flutter actúa como cliente, encargándose de la interacción con el usuario, mientras que el backend basado en microservicios (FastAPI) funciona como servidor, procesando solicitudes relacionadas con el análisis de audio y generación de retroalimentación.

Por otro lado, dentro del cliente se emplea una arquitectura en capas mediante el patrón MVVM, donde se separan claramente la capa de presentación (View), la lógica de negocio (ViewModel) y la gestión de datos (Model y Repository). Esta organización permite mejorar la mantenibilidad del sistema y facilita la escalabilidad del mismo.

Adicionalmente, el sistema incorpora un estilo orientado a eventos a través de Firebase, lo que permite la sincronización en tiempo real de los datos entre los diferentes usuarios del sistema, sin necesidad de recargar manualmente la información.

La combinación de estos estilos arquitectónicos permite construir un sistema flexible, desacoplado y capaz de responder de manera eficiente a los cambios en tiempo real.

4. Patrones arquitectónicos utilizados

En el desarrollo del sistema se emplean diversos patrones arquitectónicos que permiten organizar los componentes de manera eficiente, facilitando la escalabilidad, mantenibilidad y reutilización del código.

En primer lugar, se utiliza el patrón MVVM (Model-View-ViewModel) en la aplicación cliente desarrollada en Flutter. Este patrón permite separar la interfaz de usuario (View) de la lógica de negocio (ViewModel) y de los datos (Model). De esta manera, la actualización de la interfaz se realiza automáticamente cuando cambian los datos, mejorando la interacción con el usuario y reduciendo el acoplamiento entre componentes.

En segundo lugar, se implementa el patrón Repository, el cual actúa como una capa intermedia entre el ViewModel y las fuentes de datos. Este patrón permite centralizar el acceso a la información, ya sea que provenga de servicios externos (como el backend en FastAPI) o de bases de datos (Firebase), facilitando futuros cambios sin afectar la lógica de la aplicación.

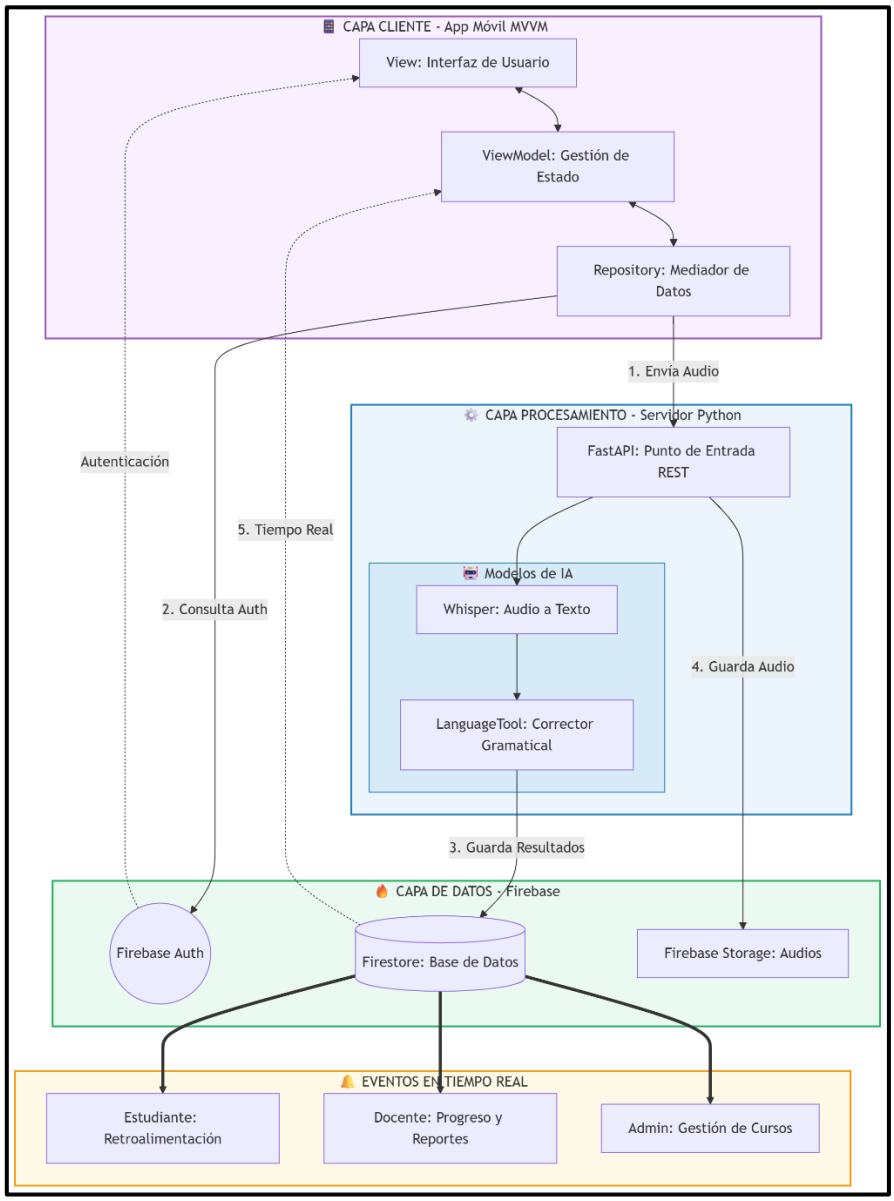
Asimismo, se emplea una arquitectura basada en microservicios, donde el backend se encuentra desacoplado de la aplicación cliente. Este microservicio, desarrollado en FastAPI, se encarga de realizar tareas específicas como el procesamiento de audio mediante herramientas de reconocimiento de voz y el análisis gramatical, permitiendo distribuir la carga de trabajo y mejorar el rendimiento del sistema.

Finalmente, se incorpora un enfoque de arquitectura orientada a eventos, implementado mediante Firebase, el cual permite que los cambios en los datos se propaguen automáticamente a los diferentes clientes. Esto posibilita la actualización en tiempo real de la información, tanto para estudiantes como para docentes, sin necesidad de intervención manual.

La integración de estos patrones permite construir un sistema modular, desacoplado y preparado para adaptarse a futuras mejoras o ampliaciones.

5. Diagrama arquitectónico

A continuación, se presenta el diagrama arquitectónico del sistema, el cual ilustra la estructura general de la aplicación, los componentes principales y la interacción entre ellos.



6. Descripción del diagrama

El diagrama arquitectónico presentado ilustra la estructura general del sistema, el cual se encuentra organizado en: cliente, procesamiento y datos, incorporando un mecanismo de eventos en tiempo real, mostrando además el flujo de información entre cada uno de sus componentes.

- En la capa cliente, desarrollada bajo el **patrón MVVM,** se identifican tres elementos principales: la View, encargada de la interfaz de usuario; el ViewModel, responsable de la gestión del estado y la lógica de presentación; y el Repository, que actúa como mediador entre la aplicación y las fuentes de datos externas. La interacción entre estos componentes permite una correcta separación de responsabilidades y una actualización dinámica de la interfaz.
- La **capa de procesamiento**, implementada mediante un servidor en Python utilizando FastAPI, funciona como punto de entrada para las solicitudes provenientes del cliente. En esta capa se integran los modelos de inteligencia artificial, donde Whisper se encarga de transformar el audio en texto, mientras que LanguageTool realiza el análisis y corrección gramatical del contenido generado.
- En la **capa de datos**, se utiliza Firebase como plataforma principal, incluyendo servicios como Firebase Auth para la autenticación de usuarios, Firestore como base de datos para el almacenamiento de resultados, y Firebase Storage para el manejo de archivos de audio. Esta capa permite centralizar la información y garantizar su disponibilidad.
- Finalmente, el sistema incorpora una **arquitectura orientada a eventos**, donde Firestore permite la actualización en tiempo real de los datos. Esto posibilita que diferentes roles del sistema, como estudiantes, docentes y administradores, reciban información actualizada de manera automática, sin necesidad de recargar la aplicación.

En cuanto al flujo de funcionamiento, el proceso inicia cuando el usuario envía un audio desde la aplicación móvil. Este es gestionado por el Repository y enviado al servidor FastAPI, donde se procesa mediante los modelos de inteligencia artificial. Posteriormente, los resultados son almacenados en Firestore y el audio en Firebase Storage. Una vez guardados los datos, se genera un evento en tiempo real que permite

actualizar automáticamente la información en la aplicación, reflejando los cambios en la interfaz del usuario correspondiente.

7. Justificación de la arquitectura

La arquitectura seleccionada para el desarrollo del sistema se basa en la integración de múltiples patrones y enfoques que permiten satisfacer de manera eficiente los requerimientos funcionales y no funcionales de la aplicación.

- En primer lugar, se adopta el patrón MVVM en la capa cliente, ya que permite una adecuada separación entre la interfaz de usuario, la lógica de presentación y la gestión de datos. Esto facilita el mantenimiento del código, mejora la escalabilidad del sistema y permite una actualización automática de la interfaz cuando cambian los datos, lo cual es fundamental para una aplicación interactiva.
- El uso del patrón Repository se justifica por la necesidad de abstraer el acceso a las distintas fuentes de datos, permitiendo que el ViewModel no dependa directamente de servicios externos como el backend o la base de datos. Esto proporciona mayor flexibilidad ante futuros cambios tecnológicos y reduce el acoplamiento entre componentes.
- Por otra parte, la implementación de una arquitectura basada en microservicios, mediante un servidor independiente desarrollado en FastAPI, permite delegar el procesamiento de tareas complejas, como el reconocimiento de voz y el análisis gramatical, a un entorno especializado. Esto mejora el rendimiento de la aplicación móvil, evitando sobrecargar el dispositivo del usuario y permitiendo escalar el sistema de manera independiente.
- Asimismo, el uso de Firebase como plataforma de datos y autenticación se justifica por su capacidad de gestionar información en tiempo real, lo que facilita la implementación de una arquitectura orientada a eventos. Gracias a este enfoque, los cambios en los datos se reflejan automáticamente en la aplicación, permitiendo que estudiantes, docentes y administradores accedan a información actualizada sin necesidad de recargar manualmente la interfaz.

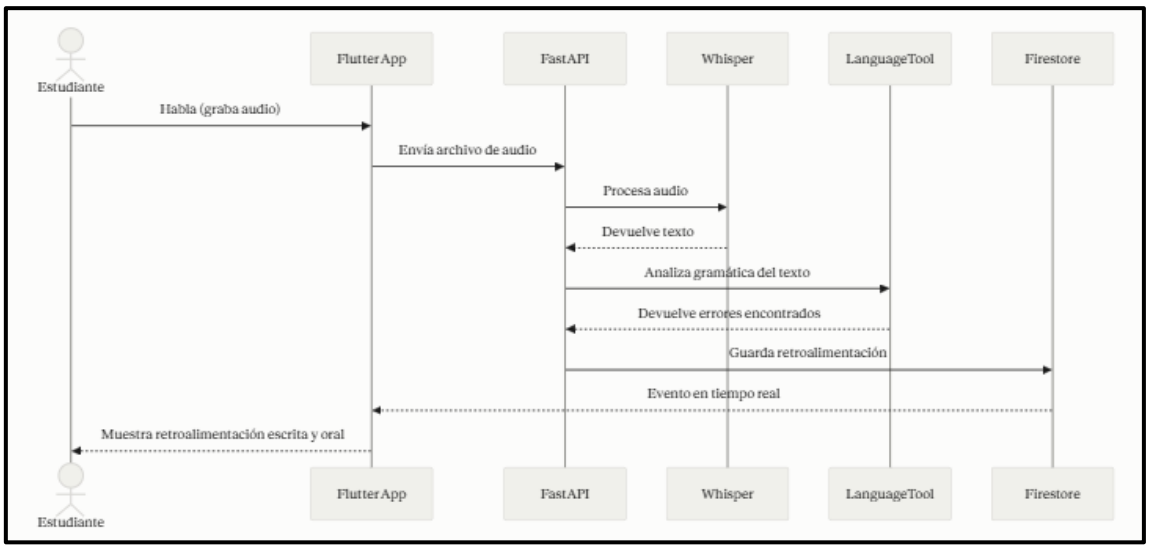
En conjunto, la combinación de estos patrones y tecnologías permite construir un sistema modular, desacoplado y eficiente, capaz de adaptarse a futuras mejoras, soportar múltiples usuarios y ofrecer una experiencia fluida en tiempo real.

8. Conclusión

El diseño arquitectónico desarrollado para la aplicación permite establecer una base sólida para su implementación, garantizando una adecuada organización de los componentes y una correcta interacción entre ellos. La integración de los patrones MVVM, Repository, microservicios y el enfoque orientado a eventos contribuye a la construcción de un sistema modular, escalable y fácil de mantener.

Asimismo, la separación de responsabilidades entre la capa cliente, la capa de procesamiento y la capa de datos facilita el desarrollo y la evolución del sistema, permitiendo incorporar mejoras futuras sin afectar significativamente su estructura. El uso de tecnologías como Firebase y FastAPI complementa esta arquitectura al proporcionar herramientas eficientes para la gestión de datos, autenticación y procesamiento especializado.

Finalmente, el enfoque adoptado permite ofrecer una experiencia de usuario dinámica y en tiempo real, lo cual resulta fundamental para el tipo de aplicación propuesta. En este sentido, el diseño arquitectónico no solo responde a las necesidades actuales del sistema, sino que también sienta las bases para su crecimiento y adaptación en entornos más complejos.

Anexo

Flujo de la Conversación